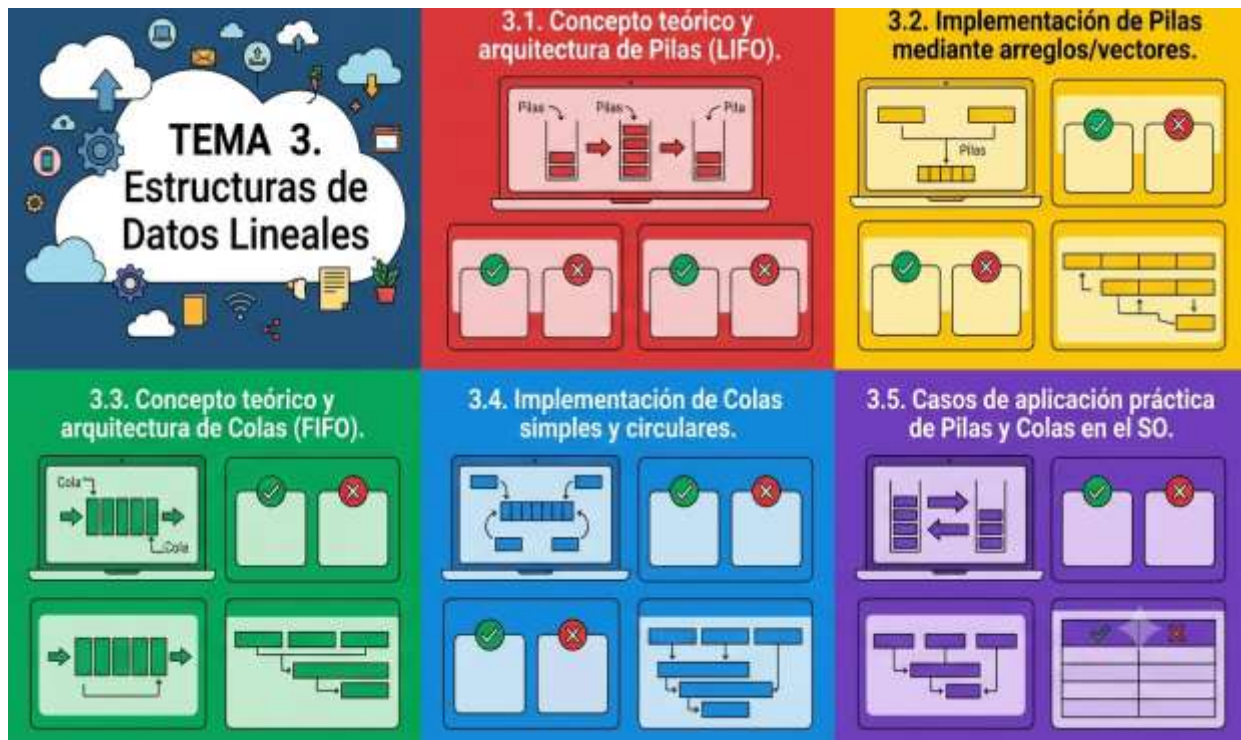


TEMA N° 3



TEMA 3. ESTRUCTURAS DE DATOS LINEALES



EN ESTE TEMA SERÁS CAPAZ DE...

1. **Comprender con precisión** el funcionamiento, la arquitectura interna y la lógica operativa de las estructuras de datos lineales de tipo Pila (LIFO) y Cola (FIFO), identificando sus diferencias clave y sus entornos de aplicación óptimos.
2. **Implementar de forma práctica** algoritmos robustos en lenguaje de programación C para construir tanto pilas como colas (simples y circulares) utilizando arreglos estáticos, aplicando buenas prácticas de control de desbordamiento de memoria.
3. **Resolver problemas tecnológicos reales** del entorno institucional y comunitario, diseñando soluciones de software eficientes que optimicen la gestión de procesos bajo restricciones de hardware en proyectos informáticos locales.



PRÁCTICA



¿Te has puesto a pensar qué sucede dentro de la memoria de un sistema informático cuando los administrativos del CEA hacen clic repetidamente en el botón "Deshacer" (Ctrl+Z) para corregir un registro de inscripción, o cómo procesa el servidor del centro las solicitudes de impresión cuando todos los docentes envían sus exámenes del trimestre al mismo tiempo?

Imaginemos una situación real en nuestro nuevo módulo educativo del CEA Herman Ortiz Camargo, ubicado en el Barrio Saó de nuestra población de Pailón. Con la entrega de la infraestructura nueva y equipada, la dirección institucional ha decidido automatizar el control de acceso y el registro de mensualidades de los estudiantes que asisten desde distintos puntos de la región, como el Barrio Central Fátima, el Barrio 24 de Septiembre y las zonas productivas circundantes.

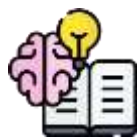
El problema surge durante la hora pico de ingreso nocturno. Los estudiantes forman una fila física en el pasillo central del módulo para registrar su asistencia y validar sus comprobantes de pago en una única computadora asignada en secretaría. Paralelamente, el sistema de red local recibe ráfagas de datos provenientes de la terminal de asistencia biométrica y peticiones de consulta de notas enviadas por los docentes desde las aulas del segundo piso.

Como Técnico en Sistemas Informáticos encargado del desarrollo, te encuentras con los siguientes desafíos técnicos en la computadora local:

1. El sistema debe almacenar temporalmente las acciones realizadas por el secretario del CEA de modo que, si comete un error registrando un pago de un estudiante de los Barrios Jardines de Pailón o Las Misiones, pueda revertir la última acción de inmediato, recuperando el estado anterior de forma exacta.
2. Las solicitudes de impresión de boletas de calificaciones enviadas por la impresora institucional compartida se están mezclando, lo que causa que las hojas de los estudiantes del Barrio 27 de Mayo salgan intercaladas con las del Barrio San José, interrumpiendo el flujo de trabajo.



A nivel de software, no podemos simplemente almacenar los datos de manera desordenada en variables comunes. Si procesamos las solicitudes de reversión de acciones en el orden incorrecto, borraremos datos válidos. Si atendemos las peticiones de impresión de forma aleatoria, generaremos un caos administrativo en el nuevo módulo. No disponemos de servidores en la nube ilimitados; estamos trabajando directamente sobre el hardware local del CEA y requerimos una estructura organizada que controle el flujo de entrada, permanencia y salida de los datos en la memoria RAM del sistema.



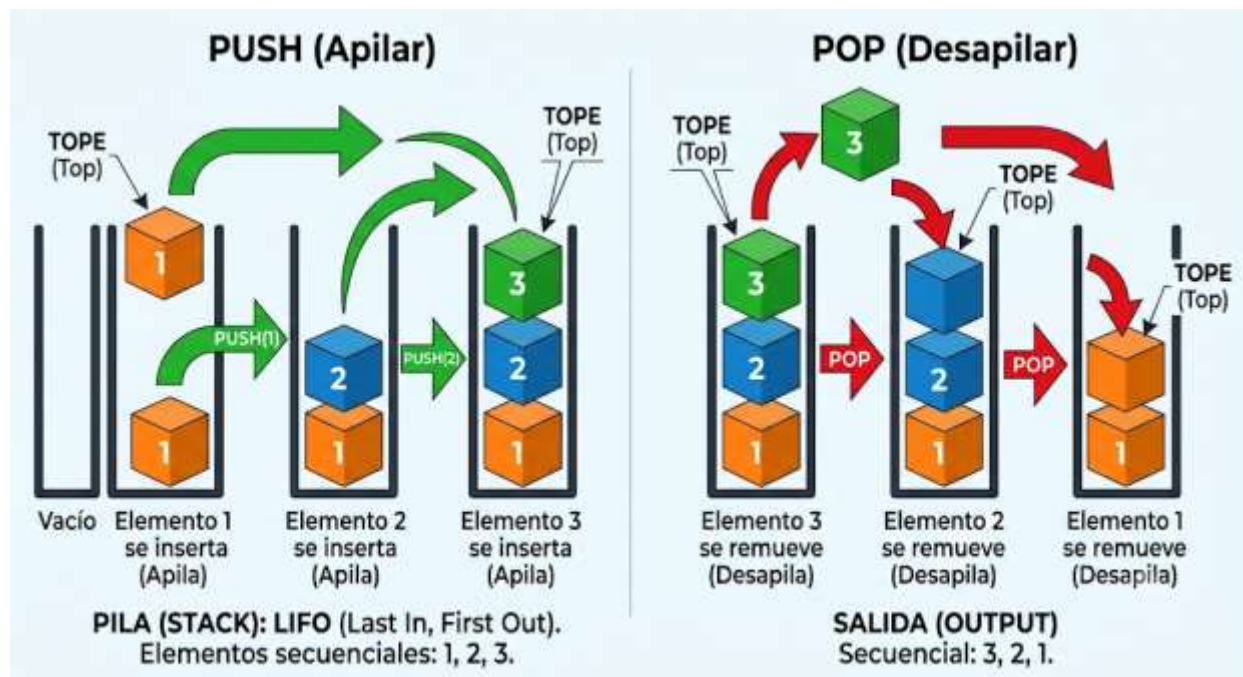
TEORÍA



3.1. CONCEPTO TEÓRICO Y ARQUITECTURA DE PILAS (LIFO)

Una Pila (conocida en inglés como *Stack*) es una estructura de datos lineal y dinámica en la que los elementos se introducen y se extraen siguiendo un principio estrictamente definido: el algoritmo **LIFO** (*Last In, First Out*), que se traduce al español como "Último en Entrar, Primero en Salir". Esto significa que el último dato que ha sido almacenado en la memoria asignada a la estructura será obligatoriamente el primer elemento en ser procesado y retirado.

Para comprenderlo de forma sencilla en nuestro contexto diario en Pailón, piensa en una pila de bolsas de cemento ordenadas verticalmente en el depósito del nuevo módulo educativo del Barrio Saó durante la fase de equipamiento. Los albañiles colocan una bolsa encima de otra. Al momento de usar el cemento para los revoques, no retiran la bolsa que está abajo en el piso, sino que toman la que está arriba de todo, es decir, la última que colocaron.



En la arquitectura de una pila en la memoria RAM, existe un único punto de acceso llamado **Tope** o **Cúspide** (*Top*). Todas las operaciones de modificación de la estructura se realizan exclusivamente a través de este puntero de control. Las operaciones fundamentales que definen el comportamiento de una pila son:

1. **Push (Apilar):** Inserta un nuevo elemento en la parte superior de la pila, incrementando la posición del indicador del tope.
2. **Pop (Desapilar):** Elimina y devuelve el elemento ubicado en el tope de la pila, decrementando la posición del indicador.
3. **Peek / Top (Mirar Cúspide):** Consulta el valor del elemento que está en el tope sin retirarlo de la estructura.
4. **IsEmpty (Esta Vacía):** Función booleana que verifica si la pila carece de elementos para evitar errores de lectura.
5. **IsFull (Esta Llena):** Función que determina si la estructura ha alcanzado el límite máximo de memoria reservada.



3.2. IMPLEMENTACIÓN DE PILAS MEDIANTE ARREGLOS/VECTORES

Para implementar una pila en lenguaje C utilizando almacenamiento estático, recurrimos a los arreglos (vectores) de tamaño fijo. Esta técnica requiere definir un tamaño máximo de almacenamiento antes de la compilación y utilizar una variable entera auxiliar que actúe como el índice del Tope. Cuando la pila está vacía, este índice se inicializa en -1, indicando que no apunta a ninguna posición válida del arreglo.

A continuación, se presenta la implementación completa y funcional en código C, estructurada de forma clara para su ejecución directa en los equipos de computación de nuestro laboratorio del CEA:

C

```
#include <stdio.h> // Incluye la librería estándar para operaciones de entrada y salida
#define MAX 5 // Define el tamaño máximo de la pila mediante una constante
int pila[MAX];
// Declara el arreglo global que almacenará los datos de la pila
int tope = -1;
// Inicializa el indicador de la cúspide en -1, indicando pila vacía
// Función para verificar si la estructura se encuentra completamente llena
int estaLlena() {
    if (tope == MAX - 1) {
        // Evalúa si el índice del tope llegó al límite del arreglo      return 1;
        // Retorna verdadero si la pila está llena      }
        // Cierre del condicional if      return 0;
        // Retorna falso si aún queda espacio disponible}
        // Función para verificar si la estructura carece de elementos cargados
        int estaVacia() {
            if (tope == -1) {
                // Comprueba si el tope se mantiene en su estado inicial      return 1;
                // Retorna verdadero si la pila no contiene datos      }
                // Cierre del condicional if      return 0;
                // Retorna falso si la pila tiene al menos un elemento}
                // Función para insertar un dato en el tope de la estructura
                void push(int dato)
                {
                    if (estaLlena() == 1) {
                        // Verifica el estado de la pila antes de insertar      printf("
                        Error: Desbordamiento de pila (Stack Overflow). No se puede apilar.\n");
                        // Alerta de falta de espacio      }
                        else {
                            // Bloque de ejecución si hay espacio disponible      tope++;
                            // Incrementa el índice del tope para apuntar a la nueva celda vacía
                            pila[tope] = dato;
```



```
// Asigna el dato recibido dentro del arreglo en la posición del tope
printf("
Elemento [%d] apilado correctamente.\n", dato);
// Muestra confirmación al usuario    }
// Cierre del condicional else}
// Función para retirar y retornar el elemento ubicado en la cúspideint pop() {
if (estaVacía() == 1) {
// Verifica si hay elementos para extraer en la estructura    printf("
Error: Subdesbordamiento de pila (Stack Underflow). Pila vacía.\n");
// Alerta de estructura vacía    return -1;
// Retorna un valor de error predeterminado    }
else {
// Bloque de ejecución si contiene datos    int valorExtraido = pila[tope];
// Guarda temporalmente el valor del tope actual    tope
--;
// Decrementa el índice para dar de baja virtual al elemento eliminado
return valorExtraido;
// Devuelve el valor que se retiró de la estructura    }
// Cierre del condicional else}
// Función principal de ejecución del programa de controlint main() {
printf("
--- Simulador de Pila para Control Administrativo del CEA -
--\n");
// Título informativo    push(101);
// Apila el registro del primer estudiante del Barrio Saó    push(102);
// Apila el registro del segundo estudiante del Barrio Central Fátima
push(103);
// Apila el registro del tercer estudiante del Barrio 24 de Septiembre
printf("
Dato retirado de la pila: %d\n", pop());
// Debe retirar el 103 por la lógica LIFO    printf("
Dato retirado de la pila: %d\n", pop());
// Debe retirar el 102 secuencialmente    return 0;
// Finaliza la ejecución del programa de forma exitosa}
```

3.3. CONCEPTO TEÓRICO Y ARQUITECTURA DE COLAS (FIFO)

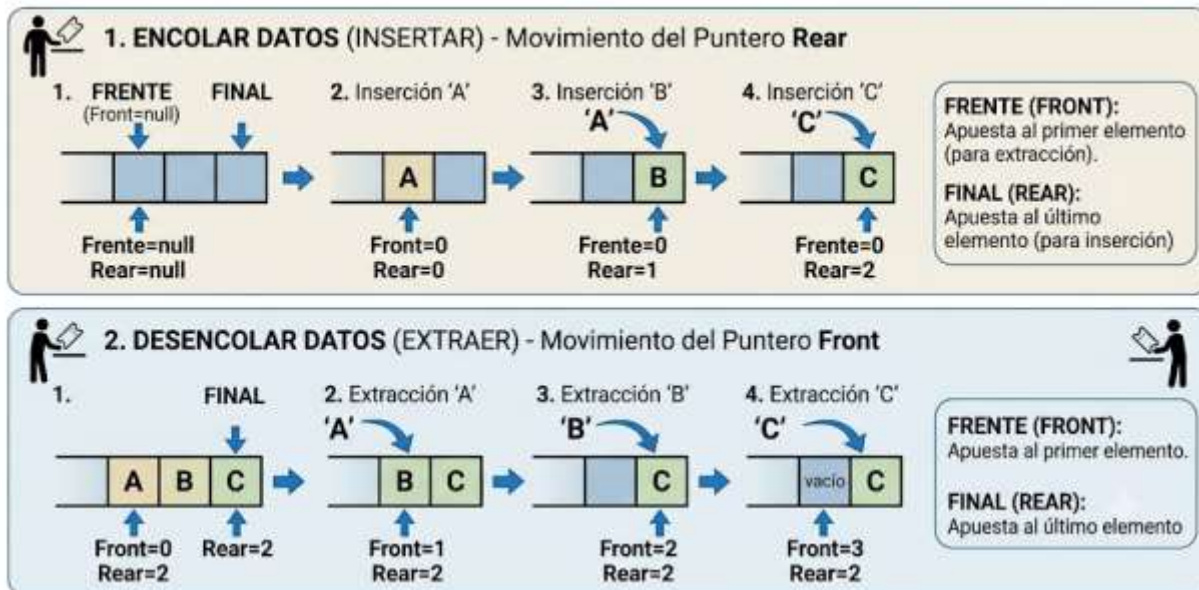
Una Cola (conocida en informática como *Queue*) es una estructura de datos lineal donde los elementos se gestionan bajo una regla inversa a las pilas: el principio **FIFO** (*First In, First Out*), que significa "Primero en Entrar, Primero en Salir". En este modelo, el primer elemento que se agrega a la cola será rigurosamente el primer elemento en ser procesado y removido de la memoria.

Para visualizarlo en nuestro entorno de Pailón, piensa en la fila que realizan los vecinos en el Barrio Central Fátima afuera de la iglesia o en las cajas de la cooperativa de agua en días de cobro. El primer vecino que llega y se sitúa al frente es atendido antes que los demás, y los nuevos



usuarios que van llegando deben colocarse obligatoriamente al final de la fila, esperando pacientemente su turno.

Arquitectura de Qempla Simplicia (FIFO) y Movimiento del Puntero Rear



A diferencia de la pila, la arquitectura de una cola simple requiere de **dos puntos de acceso** o punteros de control independientes para operar de manera óptima:

1. **Frente (Front):** Puntero que apunta al índice del elemento que está listo para ser extraído o atendido (la cabeza de la fila).
2. **Final / Atrás (Rear / Back):** Puntero que señala la última posición donde se insertó un dato (la cola de la fila).

Las operaciones básicas en una estructura de tipo cola son:

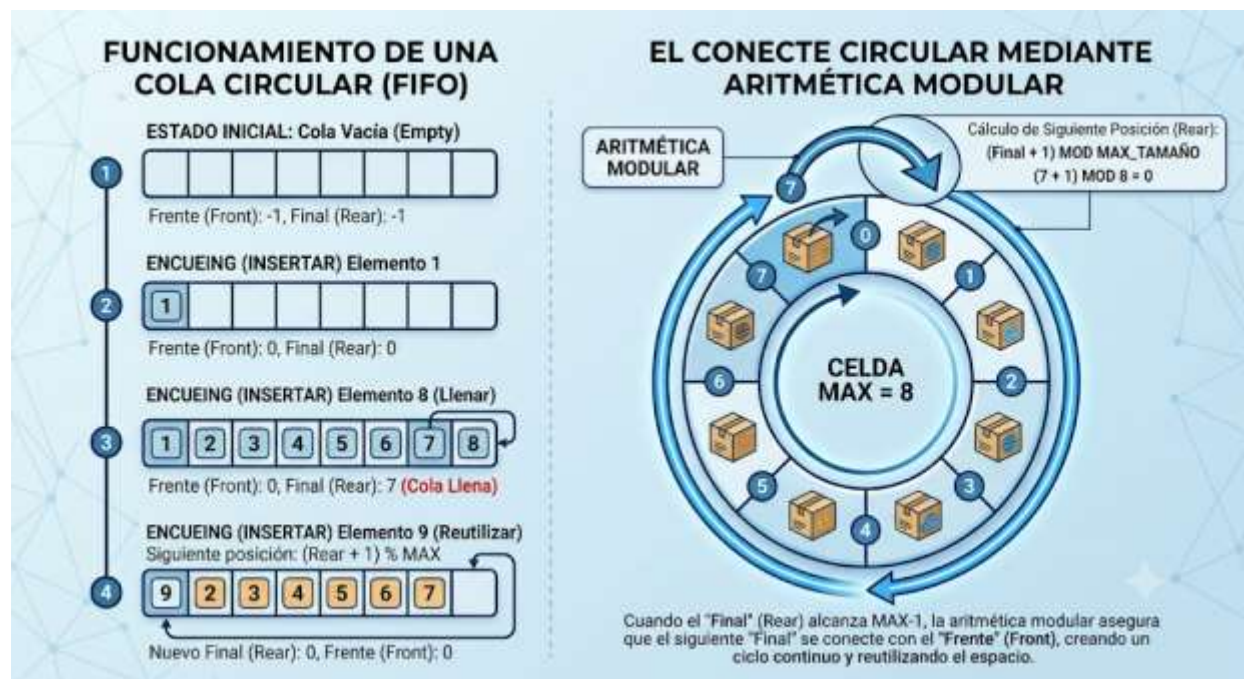
1. **Enqueue (Encolar):** Añade un nuevo elemento por el extremo final de la cola.
2. **Dequeue (Desencolar):** Extrae y procesa el elemento ubicado en el extremo del frente.
3. **Front (Consultar Frente):** Examina el valor del elemento que está primero en la fila sin eliminarlo.



3.4. IMPLEMENTACIÓN DE COLAS SIMPLES Y CIRCULARES

El mayor inconveniente al implementar una **cola simple** mediante arreglos lineales es el desperdicio de memoria RAM. A medida que realizamos operaciones de extracción (dequeue), el puntero del Frente avanza hacia la derecha. Eventualmente, el puntero Final alcanzará el último índice del arreglo ($MAX - 1$). En este punto, el sistema determinará que la cola está llena (IsFull), aunque las primeras posiciones del arreglo hayan quedado vacías debido a las extracciones previas.

Para resolver de raíz esta ineficiencia técnica sin recurrir a costosos desplazamientos de memoria, los Técnicos en Sistemas Informáticos implementamos la **Cola Circular**. En esta variante arquitectónica, el arreglo se comporta de forma cíclica, conectando conceptualmente el último casillero con el primero mediante el uso de la operación matemática de residuo o **aritmética modular** (%). De este modo, si el puntero Final llega al extremo del arreglo y se desborda, regresa automáticamente a la posición cero si esta se encuentra desocupada.



Veamos la implementación en lenguaje C de una cola circular para gestionar el flujo de impresión del CEA:



C

```

#include <stdio.h> // Librería esencial para la salida de datos en pantalla
#define MAX 4 // Tamaño reducido del arreglo circular para facilitar el análisis
lógico int cola[MAX];
// Arreglo estático donde se almacenan los datos de los procesos int frente = -1;
// Puntero de control de salida inicializado en estado vacío int final = -1;
// Puntero de control de entrada inicializado en estado vacío
// Función para determinar si la cola circular carece enteramente de datos de
void estaVacia() {
if (frente == -1) {
// Verifica la condición de vacío absoluto del puntero inicial return 1;
// Retorna verdadero si la cola no contiene ningún elemento }
// Cierre del condicional if return 0;
// Retorna falso si la cola cuenta con elementos en espera}
// Función avanzada para determinar si la estructura circular completó su
capacidad disponible int estaLlena() {
if ((final + 1) % MAX == frente) {
// Aplica aritmética modular para predecir si el extremo final chocará con el
frente return 1;
// Retorna verdadero indicando que la cola está llena }
// Cierre del condicional if return 0;
// Retorna falso si aún se cuenta con espacios libres en el ciclo}
// Función para insertar elementos en la estructura siguiendo la lógica
circular void encolar(int item) {
if (estaLlena() == 1) {
// Control preventivo de desbordamiento antes de insertar datos printf("
Error: La Cola Circular se encuentra llena. Solicitud rechazada.\n");
// Mensaje de saturación }
else {
// Si la validación es exitosa prosigue con la inserción if (frente == -
1) {
// Caso especial: Si es el primer elemento que ingresa al sistema
frente = 0;
// Mueve el frente a la posición inicial indexable }
// Cierre del condicional interno final = (final + 1) % MAX;
// Incrementa el puntero final de forma circular usando el residuo de la
división cola[final] = item;
// Guarda el dato en el casillero calculado dinámicamente printf("
Trabajo [%d] encolado en posición de memoria [%d].\n", item, final);
// Muestra datos del proceso }
// Cierre del condicional else}
// Función para remover y procesar elementos del frente de la estructura int
desencolar() {
if (estaVacia() == 1) {
// Valida que existan elementos para prevenir lecturas erróneas printf("
Error: La Cola Circular está vacía. No hay elementos a procesar.\n");
// Alerta técnica return -1;

```



```
// Retorna un código de error de procesamiento    }
else {
// Si la cola contiene elementos activos          int itemExtraido = cola[frente];
// Almacena temporalmente el valor del frente actual    if (frente == final)
{
// Condición crítica: Si la cola se queda con un solo elemento restante
frente = -1;
// Resetea el puntero frontal al estado inicial de vacío          final = -1;
// Resetea el puntero final al estado inicial de vacío          }
else {
// Si existen más elementos pendientes en la fila          frente = (frente +
1) % MAX;
// Desplaza el frente hacia el siguiente elemento aplicando lógica modular
}
// Cierre del condicional anidado          return itemExtraido;
// Retorna con éxito el valor entero recuperado de la memoria    }
// Cierre del condicional else}
// Función principal para verificar la lógica circular del programa de
ejemplo int main() {
printf("
--- Control de Cola Circular para Impresiones - CEA -
--\n");
// Título explicativo          encolar(501);
// Registra impresión del reporte de notas del Barrio 27 de Mayo
encolar(502);
// Registra impresión del Listado del Barrio Residencial Norte          encolar(503);
// Registra impresión de actas del Barrio María Belén          printf("
Procesando trabajo: %d\n", desencolar());
// Desencola el primer archivo ingresado: 501          encolar(504);
// Inserta un nuevo trabajo aprovechando la naturaleza de la cola circular
encolar(505);
// Comprobación de retorno al índice inicial gracias a la aritmética modular
return 0;
// Finaliza la ejecución del script limpiamente}
```

3.5. CASOS DE APLICACIÓN PRÁCTICA DE PILAS Y COLAS EN EL SO

A nivel de Sistemas Operativos (SO) y de arquitectura de software que un Técnico en Sistemas Informáticos debe dominar, estas dos estructuras de datos lineales sostienen la mayor parte de las operaciones automatizadas esenciales del hardware:

1. **Pila de Llamadas del Sistema (Call Stack):** El sistema operativo utiliza una pila LIFO para gestionar la ejecución de funciones y subrutinas dentro de cualquier software instalado. Cuando una función llama a otra, el sistema operativo apila la dirección de memoria de retorno y las variables locales. Al terminar la ejecución de la subtarea, realiza un pop para



regresar exactamente al punto donde el usuario se había quedado. Es lo que permite que programas complejos funcionen de manera fluida y secuencial en las computadoras de escritorio del CEA.

2. **Mecanismos de Reversión (Undo / Redo):** Editores de texto, herramientas de diseño gráfico y hojas de cálculo utilizan internamente una pila para guardar los cambios en el documento, permitiendo al usuario volver atrás en el tiempo con precisión matemática.
3. **Planificación del Procesador (CPU Scheduling):** Los sistemas operativos multitarea gestionan el tiempo de ejecución del microprocesador mediante colas FIFO. Los procesos que los usuarios inician entran en una cola de "Listos" esperando que el procesador les asigne ciclos de reloj. El algoritmo *Round Robin* utiliza colas para dar tiempos de ejecución equitativos a cada aplicación.
4. **Colas de Buffer (Almacenamiento Intermedio):** La transmisión de paquetes de datos a través de la red local del módulo educativo hacia los enrutadores de internet, así como la transferencia de bytes hacia dispositivos lentos (como memorias flash USB o impresoras), se realiza mediante buffers estructurados estrictamente como colas para evitar pérdidas de información.

3.6. ESTRUCTURAS DE DATOS LINEALES EN LA ERA DE LOS MICROSERVICIOS Y MENSAJERÍA ASÍNCRONA

En el desarrollo de software moderno y arquitectura en la nube, las colas han evolucionado desde simples estructuras de memoria local hacia robustos sistemas independientes conocidos como **Brókers de Mensajería** o Colas de Mensajes Asíncronas (como *RabbitMQ*, *Apache Kafka* o *Amazon SQS*).

Cuando diseñamos aplicaciones empresariales fragmentadas en microservicios, estos componentes no se comunican de forma directa o síncrona, ya que si un servicio web falla, arrastraría a toda la infraestructura en un efecto dominó. En su lugar, se implementan colas distribuidas basadas en una arquitectura FIFO avanzada. Por ejemplo, en una plataforma de comercio electrónico de gran escala, cuando un usuario confirma una compra, los datos del pedido ingresan inmediatamente a una cola de mensajería asíncrona. El microservicio encargado



de procesar los cobros extrae el mensaje del frente de la cola a su propio ritmo de procesamiento, garantizando que el sistema jamás sufra caídas por picos de tráfico inesperados, elevando sustancialmente la tolerancia a fallos del ecosistema informático.

3.7. PILAS DE EJECUCIÓN EN MOTORES DE INTELIGENCIA ARTIFICIAL Y ANÁLISIS DE SINTAXIS

La revolución contemporánea de la Inteligencia Artificial (IA) y el Procesamiento del Lenguaje Natural (PLN) que da vida a los Grandes Modelos de Lenguaje (LLMs) se apoya en sus raíces en la optimización del análisis sintáctico, una tarea clásica gobernada por las pilas. Los compiladores modernos e intérpretes de código utilizan pilas para validar que las instrucciones escritas por el programador respeten la sintaxis del lenguaje mediante la construcción de Árboles de Sintaxis Abstracta (AST).

Al evaluar expresiones complejas o tokens de comandos que alimentan las redes neuronales artificiales, las ecuaciones matemáticas se transforman internamente a notación postfija (Polaca Inversa), eliminando los paréntesis. Los motores de inferencia recorren los tokens de izquierda a derecha: si encuentran un operando lo apilan inmediatamente mediante un push, y al detectar un operador matemático ejecutan operaciones de pop sobre los valores superiores para procesarlos. Esta técnica limpia de manipulación lineal es la que provee la base matemática de velocidad que requieren las interfaces inteligentes modernas para responder en tiempo real.

3.8. OPTIMIZACIÓN DE ESTRUCTURAS DE DATOS LINEALES PARA INTERNET DE LAS COSAS (IOT) Y SISTEMAS EMBEBIDOS

El auge de las ciudades inteligentes y los proyectos de domótica e Internet de las Cosas (IoT) demanda del Técnico un manejo quirúrgico de la memoria RAM. Los microcontroladores de bajo costo y recursos extremadamente limitados (como los chips *ESP32* o *Arduino*) no poseen gigabytes de memoria dinámica para almacenar historiales continuos de datos recopilados por sensores meteorológicos o de automatización agrícola.

En estos entornos críticos, la **Cola Circular Estática** es la estructura reina de la optimización de código de firmware. Al conectar un sensor que mide la temperatura ambiente de un aula técnica en intervalos de segundos, los datos recolectados se insertan en una cola circular de tamaño fijo



preestablecido en el hardware. Una vez que la cola circular se completa, los nuevos datos del sensor sobrescriben los registros más antiguos de forma automatizada por diseño arquitectónico de software. Esto erradica por completo la necesidad de invocar funciones de reasignación dinámica de memoria (malloc / free), previniendo fugas de memoria (*memory leaks*) que congelarían o dañarían los dispositivos embebidos que operan de forma aislada en zonas remotas de producción.

CIERRE TEÓRICO OBLIGATORIO

❑ *Mitos vs. Realidades*

Mito Tecnológico Común	Realidad Técnica Demostrable
Las colas simples implementadas con arreglos lineales básicos aprovechan al máximo el espacio físico del vector de memoria RAM.	Falso. Las colas simples sufren de fragmentación y desperdicio de memoria conforme los elementos se eliminan, requiriendo obligatoriamente el diseño de colas circulares para reutilizar las celdas iniciales.
Las estructuras lineales como pilas y colas quedaron obsoletas ante la llegada de bases de datos relacionales y herramientas modernas de IA.	Falso. Tanto las bases de datos como los modelos de inteligencia artificial utilizan internamente pilas de ejecución y colas de procesos en su arquitectura núcleo para organizar los hilos de hardware de forma eficiente.

❑ *Glosario de Términos*

1. **LIFO (Last In, First Out):** Algoritmo de gestión de flujo de datos donde el último elemento almacenado en la memoria es restrictivamente el primero en ser extraído.
2. **FIFO (First In, First Out):** Mecanismo de control lineal donde el primer dato en ingresar a la estructura de almacenamiento conserva la prioridad absoluta de salida.
3. **Aritmética Modular:** Operación matemática basada en el cálculo del residuo de una división entera, empleada en desarrollo de software para forzar a un índice a ciclar dentro de los límites de un arreglo.



4. **Desbordamiento de Pila (Stack Overflow):** Error crítico de software que ocurre cuando un programa intenta insertar datos en una pila que ya ha completado su capacidad máxima asignada.
5. **Subdesbordamiento (Underflow):** Error operativo que se genera cuando un algoritmo intenta extraer datos o ejecutar una limpieza sobre una estructura lineal que se encuentra vacía.

□ *Ponte a Prueba*

1. Si aplicamos consecutivamente las operaciones: push(A), push(B), pop(), push(C) sobre una pila inicialmente vacía, ¿cuál es el elemento que queda posicionado actualmente en el tope de la estructura?
 1. A) El elemento A
 2. B) El elemento B
 3. C) El elemento C
2. ¿Cuál es el principal beneficio arquitectónico de implementar una Cola Circular en lugar de una Cola Simple sobre un arreglo de memoria estático?
 1. A) Permite almacenar tipos de datos completamente diferentes en el mismo arreglo.
 2. B) Soluciona el desperdicio de memoria reutilizando las posiciones iniciales mediante el operador de residuo.
 3. C) Duplica la velocidad física de procesamiento del microprocesador del servidor local.
3. En los sistemas operativos modernos, ¿qué componente de bajo nivel utiliza de manera nativa la lógica lineal FIFO para su correcto funcionamiento?
 1. A) El desfragmentador del disco duro magnético de almacenamiento.
 2. B) El sistema de gestión y priorización de hilos de la cola de impresión compartida del sistema.
 3. C) El algoritmo de encriptación de seguridad de contraseñas locales del administrador.



Respuestas correctas de control de lectura para autoevaluación: 1: C, 2: B, 3: B.



VALORACIÓN



Como futuros Técnicos en Sistemas Informáticos egresados de las nuevas y modernas aulas de nuestro CEA Herman Ortiz Camargo en Pailón, el diseño de software no debe limitarse únicamente a escribir código veloz que compile sin errores; debe estar acompañado de una profunda **conciencia ética, social y medioambiental**.

El mal uso o la elección incorrecta de las estructuras de datos dentro de un desarrollo de software local tiene consecuencias reales en el mundo físico. Cuando un desarrollador escribe programas ineficientes que saturan la memoria RAM debido al uso descuidado de colas simples lineales que desperdician posiciones de almacenamiento, obliga a los equipos de computación a realizar un esfuerzo de hardware excesivo, elevando el consumo de energía eléctrica y sobrecalentando los componentes electrónicos de los servidores locales.

Peor aún, el software mal optimizado causa la ralentización artificial de las computadoras corporativas o de uso institucional. Esto acelera la obsolescencia tecnológica programada: las instituciones públicas y las empresas locales desechan hardware que todavía es útil simplemente porque los nuevos programas y sistemas de gestión de datos están mal programados y exigen más recursos de los estrictamente necesarios. Este descarte masivo incrementa directamente la acumulación de basura electrónica (*e-waste*), liberando metales pesados contaminantes que degradan los suelos y acuíferos de regiones altamente vulnerables y productivas de nuestra provincia Chiquitos y el departamento de Santa Cruz.

A nivel ético y de seguridad social, el control estricto de las estructuras de tipo pila es el pilar que sostiene la **privacidad de los datos de los usuarios**. Uno de los vectores de ataque cibernético más peligrosos y explotados a nivel mundial por los piratas informáticos es el denominado "Ataque de Desbordamiento de Búfer" o *Buffer Overflow*. Cuando un programador no implementa validaciones como las funciones `estaLlena()` mostradas previamente en la teoría, un atacante malintencionado puede forzar la inyección de una cantidad excesiva de datos de



entrada dentro de la pila de memoria del sistema operativo. Esto altera la dirección de retorno de la pila de llamadas, logrando que la computadora ejecute código malicioso ajeno al sistema.

Si este fallo de seguridad ocurre en un sistema administrativo institucional, los datos confidenciales de inscripciones, registros personales de filiación de los estudiantes y comprobantes de pago de mensualidades de las familias pailoneñas pueden filtrarse o ser alterados ilegalmente. Como profesionales de la computación, dominar la arquitectura interna de las estructuras de datos lineales es un acto de responsabilidad civil para proveer soluciones de software seguras, sostenibles y altamente confiables que protejan a los ciudadanos de nuestra comunidad.



PRODUCCIÓN



LABORATORIO PRÁCTICO PASO A PASO: SISTEMA AUTOMATIZADO DE TURNOS PARA SECRETARÍA DEL CEA

Objetivo del Reto: Diseñar y compilar un sistema de software de consola en lenguaje C que simule el dispensador automático de turnos de la secretaría en el nuevo módulo del CEA Herman Ortiz Camargo. El sistema debe gestionar de forma circular y óptima las solicitudes de turnos presentadas por los estudiantes que provienen de los barrios alejados como el Barrio Las Misiones y el Barrio San José, evitando cualquier tipo de desperdicio de memoria RAM.

Instrucciones Técnicas de Implementación:

1. Cree un nuevo archivo de código fuente en su entorno de desarrollo preferido (como Code::Blocks, Dev-C++ o VS Code) y guárdelo con el nombre descriptivo de control_turnos_cea.c.
2. Implemente la arquitectura de una **Cola Circular** utilizando arreglos estáticos con un tamaño controlado de 5 casilleros, simulando que el sistema atiende en ráfagas de alta demanda administrativa.



3. Copie de forma exacta el código fuente de producción provisto abajo, analizando detalladamente los comentarios informativos colocados en cada una de las líneas para comprender su funcionamiento interno.

C

```
#include <stdio.h> // Incluye la librería estándar indispensable para el flujo
de interacción de texto
#define TAMANO 5 // Define una capacidad máxima de 5 turnos concurrentes en el
sistema de gestión
int turnos[TAMANO];
// Arreglo estático de enteros para almacenar los códigos de turnos generados
int head = -1;
// Inicializa el indicador de la cabeza de la cola circular en estado
inactivo
int tail = -1;
// Inicializa el indicador del extremo final de la cola circular en estado
inactivo
// Función técnica encargada de validar si el sistema se quedó sin cupos libres
para turnos
int verificarSaturacion() {
if ((tail + 1) % TAMANO == head) {
// Comprobación matemática de colisión de índices de la cola circular
return 1;
// Devuelve 1 denotando que el sistema está completamente lleno }
// Fin del bloque condicional return 0;
// Devuelve 0 si existen casilleros disponibles para registrar nuevos turnos}
// Función encargada de determinar si la cola de turnos se encuentra
completamente desocupada
int verificarVacio() {
if (head == -1) {
// Verifica la posición por defecto del marcador de cabecera del arreglo
return 1;
// Retorna verdadero confirmando la ausencia total de solicitudes en memoria
}
// Fin del bloque condicional return 0;
// Retorna falso si hay usuarios en espera dentro de la fila circular}
// Función para registrar un nuevo turno en el sistema de atención del CEA
void emitirTurno(int codigoEstudiante) {
if (verificarSaturacion() == 1) {
// Llama a la validación de control para evitar fallos de desbordamiento
printf("[⚠ ALERTA] Sistema saturado en secretaría. Espere a que se despachen
turnos previos.\n");
// Mensaje de advertencia }
else {
// Si la validación es libre procede con la asignación if (head == -1) {
// Evalúa si es el primer turno procesado en el inicio del día administrativo
head = 0;
// Mueve el índice de cabecera a la casilla inicial indexable 0 }
// Fin de la condición inicial tail = (tail + 1) % TAMANO;
// Calcula de forma circular el nuevo índice de inserción usando aritmética
```



```

modular      turnos[tail] = codigoEstudiante;
// Guarda el identificador numérico en la celda calculada de la memoria
printf("[✓REGISTRO] Turno para Estudiante ID: %d asignado con exito en posicion
[%d].\n", codigoEstudiante, tail);
// Confirmación en consola      }
// Fin del bloque condicional else}
// Función para llamar y despachar al siguiente estudiante en espera desde la
secretaría void atenderTurno() {
if (verificarVacio() == 1) {
// Control preventivo para evitar procesar celdas con datos inválidos o nulos
printf("[⚠ INFORMACION] No existen estudiantes en la cola de espera de
secretaria.\n");
// Mensaje informativo      }
else {
// Si existen turnos registrados avanza el despacho      int
estudianteAtendido = turnos[head];
// Recupera el código del estudiante ubicado en la cabeza actual de la cola
printf("[⚠ ATENCION] Secretaria Llama al Estudiante con ID: %d para su atencion
inmediata.\n", estudianteAtendido);
// Llamada formal      if (head == tail) {
// Validación especial: Si se acaba de despachar el último turno activo de la
cola      head = -1;
// Reestablece el marcador de cabecera a su estado nulo original      tail
= -1;
// Reestablece el marcador final a su estado nulo original      printf("[⚠
LIMPIEZA] Cola circular vaciada por completo y reseteada para nuevos
registros.\n");
// Notificación técnica      }
else {
// Si aún restan más personas en espera detrás del estudiante atendido
head = (head + 1) % TAMANO;
// Hace avanzar el puntero de la cabeza de forma circular al siguiente casillero
}
// Fin del condicional anidado de control de índices      }
// Fin del bloque condicional else}
// Función principal para la simulación automatizada de las acciones del
laboratorio diario int main() {
printf("=== SISTEMA DE GESTION DE TURNOS AUTOMATIZADOS - CEA HERMAN ORTIZ
CAMARGO ===\n\n");
// Encabezado de la interfaz de consola      // Simulación del ingreso
secuencial de estudiantes desde diferentes locaciones      emitirTurno(20261);
// Llega estudiante del Barrio Las Misiones al nuevo módulo educativo
emitirTurno(20262);
// Llega estudiante del Barrio San José solicitando certificados de notas
emitirTurno(20263);
// Llega estudiante del Barrio Saó a convalidar sus requisitos de inscripción
emitirTurno(20264);
// Llega estudiante del Barrio Central Fátima a consultar horarios de clases
printf("\n

```



```
--- INICIO DE ATENCION EN VENTANILLA DE SECRETARIA -
--\n");
// Separador visual    atenderTurno();
// Atiende secuencialmente al primer estudiante registrado (ID: 20261)
atenderTurno();
// Atiende secuencialmente al segundo estudiante registrado (ID: 20262)
printf("\n
--- COMPROBACION DE LA NATURALEZA CIRCULAR DE LA ESTRUCTURA -
--\n");
// Separador visual    emitirTurno(20265);
// Inserta un nuevo estudiante aprovechando las celdas liberadas al inicio
emitirTurno(20266);
// Comprobación del ciclo completo regresando al índice cero de la memoria
printf("\n
--- PROCESAMIENTO DE LOS TURNOS RESTANTES -
--\n");
// Separador de operaciones finales    atenderTurno();
// Despacha de forma ordenada al estudiante con ID: 20263    atenderTurno();
// Despacha de forma ordenada al estudiante con ID: 20264    atenderTurno();
// Despacha de forma ordenada al estudiante con ID: 20265    atenderTurno();
// Despacha de forma ordenada al estudiante con ID: 20266    return 0;
// Finaliza la ejecución del entorno demostrativo de manera exitosa y limpia}
```

1. Compile y ejecute el código en la computadora de su aula. Observe detalladamente cómo los mensajes impresos en la consola demuestran que los índices se reutilizan cíclicamente gracias al operador de residuo matemáticamente controlado, cumpliendo con el estándar de una cola circular profesional.

RECURSOS ADICIONALES

1. **Documentación de Referencia del Estándar del Lenguaje ISO C (cppreference.com):** Consulta las guías técnicas oficiales sobre la gestión y declaración de arreglos estáticos de una y más dimensiones, el alcance de las variables globales y el comportamiento del operador de residuo matemático aritmético.
2. **Repositorio de Algoritmos y Estructuras de Datos Clásicas en GitHub:** Explora implementaciones avanzadas de estructuras lineales escritas por ingenieros y desarrolladores senior de la comunidad de código abierto, analizando el uso de estructuras de datos complejas aplicadas a entornos reales.
3. **Manual de Buenas Prácticas del Programador para Sistemas Embebidos y Control de Memoria:** Recomendaciones y pautas internacionales de optimización de código para



evitar fugas de memoria y desbordamientos en microcontroladores y computadoras industriales de bajos recursos de hardware.